STREAMWORKS
WORKLOAD AUTOMATION

Detailed Design Document - CR 24850

Physikalisches Löschen von Agenten

www.it.arvato.com/streamworks

ARVATO SYSTEMS

Table of contents

Table of contents **1**

1 Document **2**

Document information 2

Document history 2

2 CR Description **3**

2.1 Overview 3

2.2 Out of scope 3

2.3 Definition of terms 3

3 Implementation details **4**

3.1 Database 4

3.2 GUI 4

3.3 Backend 4

3.3.1 General 4

3.3.2 Processing Server 4

3.3.3 Central Processing Services 4

3.3.4 AppServer 4

3.4 Utilities 4

3.4.1 Export/Import 4

3.4.2 Accounting 4

3.4.3 Reorg 4

3.5 PSL 4

3.6 Streamworks REST API 5

3.6.1 New or modified methods 9

3.6.2 Default return codes 10

3.6.3 List of possible agent deletion errors 11

3.6.4 System Configuration Values 12

3.7 CLI 12

3.8 Setup 12

4 Tests **13**

4.1 Checklist 13

4.2 Developer- and system integration (SIT) tests 13

5 Open questions and problems **14**

6 Checklist for modifications **15**

7 Work package planning **16**

1 Document

Document information

Eigenschaft	Wert
Created at	Tuesday, 4. March 2025 13:55:00
Created by	niels.wiechmann@bertelsmann.de
Last modified at	Thursday, 11. December 2025 22:38:00
Last modified by	Föls, Mathies
Number of savings	70

Table 1 Document information

Document history

Date	Modified by	Comment
2025-06-13	wiec010	Initial document
2025-07-21	Sta69	Erster Review, Kommentare eingefügt, Telefonat mit Niels Wiechmann
2025-10-30	Foel008	Anpassung Businessexception, Liste mit Fehlern, Liste mit SystemConfiguration Einträgen
2025-11-03	Sta69	Review durchgeführt. Der Review gilt als abgeschlossen, wenn die restlichen drei Kommentare integriert worden sind. Erledigte Kommentare wurden von mir mit einem ‚Daumen hoch' markiert
2025-11-05	Foel008	Offene Kommentare bearbeitet

Table 2 Document history

2 CR Description

2.1 Overview

Streamworks ermöglicht bereits das logische Löschen von Agenten, die nur noch in der Runtime verwendet werden. Mit diesem CR soll die Löschfunktionalität dahingehend erweitert werden, dass die Agenten auch physikalisch gelöscht werden können, nachdem auch aus der Runtime heraus keine Referenzen mehr bestehen.

Das Löschen der bereits logisch gelöschten Agenten soll manuell per Streamworks REST API erfolgen.

2.2 Out of scope

Nicht vorgesehen ist ein zeitgesteuertes, automatisches Löschen. Ebenso ist keine GUI-Integration vorgesehen.

Die neue Funktionalität löscht ausschließlich die Agenten aus der Streamworks Datenbank, das Löschen der Agenten von ihrem Host ist kein Bestandteil.

2.3 Definition of terms

3 Implementation details

Ziel dieses CRs ist es, die Datenbank von bereits logisch gelöschten und mittlerweile auch verwaisten Agenteneinträgen zu bereinigen. Die Bereinigung wird über die Streamworks REST API aufgerufen.

Logisch gelöschte Agenten, die noch von anderen Entitäten referenziert werden, können nicht gelöscht werden. Eine Löschung kann erst erfolgen, wenn die referenzierenden Einträge, wie z. B. Execution-Logs nach Ende ihrer Aufbewahrungsfrist entfernt wurden. Dass ein Agent aufgrund noch existierender Referenzen nicht gelöscht werden kann, ist damit erwartbares Verhalten.

Weiterhin muss damit gerechnet werden, dass eine größere Anzahl zu löschende Agenten vorhanden seien kann, insbesondere bei der initialen Ausführung oder nach längerer Nichtnutzung der Funktionalität.

3.1 Database

Die Funktionalität soll zwar Einträge aus der Datenbank entfernen, Änderungen am Schema oder Migrationen sind jedoch nicht notwendig.

3.2 GUI

Keine Anpassungen nötig.

3.3 Backend

Die Löschung wird im Backend ausgeführt.

3.3.1 General

3.3.2 Processing Server

3.3.3 Central Processing Services

3.3.4 AppServer

3.4 Utilities

3.4.1 Export/Import

3.4.2 Accounting

3.4.3 Reorg

3.5 PSL

3.6 Streamworks REST API

Das physikalische Löschen wird über die Streamworks REST API aufgerufen. Dabei ist zu berücksichtigen, dass nicht jeder bereits logisch gelöschte Agent auch physikalisch gelöscht werden kann, und dass eine zu lange Laufzeit des Lösch-Requests (durch eine hohe Anzahl an zu löschenden Agenten) verhindert werden muss.

Daher wird für das Löschen ein Paging umgesetzt, dass es erlaubt, eine bestimmte Anzahl an Agenten (z. B. 50) mit einem Aufruf zu löschen, sollten mehr Agenten vorhanden sein, ist also ein erneuter Aufruf notwendig. Um auszuschließen, dass bei mehreren Aufrufen immer wieder versucht wird, dieselben nicht löschbaren Agenten zu löschen, kann zusätzlich ein sog. Cursor übergeben werden, der den ersten zu löschenden Agenten identifiziert. Somit kann seitenweise selbst über eine große Anzahl an Agenten iteriert werden, ohne versuchen zu müssen, denselben Agenten mehrfach zu löschen.

Weiterhin wird jeder Löschversuch in einer eigenen Transaktion ausgeführt und bei einem Fehler eine individuelle Fehlermeldung zurückgegeben.

Um dem Aufrufer Transparenz darüber zu ermöglichen, welche logisch gelöschten Agenten noch vorhanden sind, wird eine zweite REST-Methode implementiert, die eine Liste dieser Agenten zurückgibt. Die Information, ob der Agent tatsächlich löschbar ist, kann hier aber nicht bereitgestellt werden.

Die Operationen arbeiten grundsätzlich auf Ebene eines Mandanten, die Mandanten-ID muss dabei immer als Pfad-Parameter in der URL übergeben werden.

Die Agenten werden immer aufsteigend nach ID sortiert und nur auf diejenigen mit dem Status Deleted gefiltert. Damit erscheinen die ältesten Agenten mit der höchsten Wahrscheinlichkeit, erfolgreich gelöscht werden zu können, immer als erstes in der Liste. Der Cursor ist ein string, der die Base64 enkodierte ID des Agenten darstellt, und somit fest mit dem Agenten verbunden ist.

Das Löschen per Übergabe von Count und Cursor ist notwendig, um zum einen Timeout-Fehler zu verhindern und zum anderen – auch für den Aufrufer – aufwändigere asynchrone Requests unnötig zu machen. Es schafft zusätzlich die Flexibitität, gezielt auch einzelne Agenten zu löschen, in dem Count = 1 gesetzt wird.

Beispiel:

Es existieren 5 bereits logisch gelöschte Agenten Agent01 – Agent05, Agent01 und Agent03 können aktuell nicht gelöscht werden:

```
GET  /api/deletedagents/{mandatorCode}
{
  “data”:
  [
    {
      “agentName”: “Agent01”,
      “cursor”: “aaa”
    },
    {
      “agentName”: “Agent02”,
      “cursor”: “bbb”
    },
    {
      “agentName”: “Agent03”,
      “cursor”: “ccc”
    },
    {
      “agentName”: “Agent04”,
      “cursor”: “ddd”
    },
    {
      “agentName”: “Agent05”,
      “cursor”: “eee”
    }
  ],
  “totalCount”: 5
}
```

Im Beispiel sollen maximal zwei Agenten pro Request gelöscht werden, ohne die Angabe eines Cursors wird automatisch am Beginn der Liste begonnen:

```
DELETE /api/deletedagents/{mandatorCode}?count=2
{
  “results”:
  [
    {
      “cursor”: “aaa”,
      “agentName”: “Agent01”,
      “deleted”: false,
      “error”: “AgentReferredByStreamRun”,
    },
    {
      “cursor: “bbb”,
      “agentName”: “Agent02”
      “deleted”: true
    },
  ],
  “hasErrors”: true,
  “nextCursor”: “ccc”
}
```

Agent02 konnte erfolgreich physikalisch gelöscht werden, Agent01 nicht, da er noch durch einen Stream Run referenziert wird (Weitere mögliche Fehler siehe 3.6.3). Mit dem Wert für den nächsten Cursor kann ein weiterer Request abgeschickt werden:

```
DELETE /api/deletedagents/{mandatorCode}?from=ccc&count=2
{
  “results”:
  [
    {
      “cursor”: “ccc”,
      “agentName”: “Agent03”,
      “deleted”: false,
      “error”: “AgentReferredByStreamRun”,
    },
    {
      “cursor: “ddd”,
      “agentName”: “Agent04”
      “deleted”: true
    },
  ],
  “hasErrors”: true,
  “nextCursor”: “eee”
}
```

Dieser Prozess wird so lange fortgesetzt, bis einmal über alle Agenten in der Liste iteriert worden und der Wert für den nächsten Cursor null ist:

```
DELETE /api/deletedagents/{mandatorCode}?from=eee&count=2
{
  “results”:
  [
    {
      “cursor”: “eee”,
      “agentName”: “Agent05”,
      “deleted”: true
    }
  ],
  “hasErrors”: false,
  “nextCursor”: null
}
```

Eine erneute Abfrage der logisch gelöschten Agenten gibt jetzt nur noch die beiden aktuell nicht löschbaren Agenten zurück:

```
GET  /api/deletedagents/{mandatorCode}
{
  “data”:
  [
    {
      “agentName”: “Agent01”,
      “cursor”: “aaa”
    },
    {
      “agentName”: “Agent04”,
      “cursor”: “ddd”
    }
  ],
  “totalCount”: 5
}
```

Mit einem Count von 1 und dem Cursor kann später z. B. auch versucht werden, gezielt nur einen einzelnen Agenten zu löschen, was in diesem Beispiel zu einem späteren Zeitpunkt auch erfolgreich ist:

```
DELETE /api/deletedagents/{mandatorCode}?from=aaa&count=1
{
  “results”:
  [
    {
      “cursor”: “aaa”,
      “agentName”: “Agent01”,
      “deleted”: true
    }
  ],
  “hasErrors”: false,
  “nextCursor”: “ddd”
}
```

3.6.1 New or modified methods

3.6.1.1 New: GetAllLogicalDeletedAgents

Gibt eine Liste aller logisch gelöschter Agenten mit Namen und Cursor zurück. Die Liste wird nicht gepaged, daher wird ein Limit von 2000 Agenten implementiert, dass über die SystemConfiguration-Tabelle geändert werden kann.

Die Response erhält darüber hinaus die Gesamtanzahl aller logisch gelöschten Agenten.

Request:

```
GET  /api/deletedagents/{mandatorCode}
```

Response:

```
{
  “data”:
  [
    {
      “agentName”: <string>,
      “cursor”: <string>
    }
  ],
  “toatlCount”: <number>
}
```

- Default Return Code: HTTP 200 OK
- Authentifizierung: Standard-Authentifizierung (Nutzername, Passwort und API-Key)
- Autorisierung: Erfordert entweder die Rolle System-Admin oder Mandanten-Admin oder das Function-Right: ReadAgent

3.6.1.2 New: DeleteLogicalDeletedAgents

Versucht die angegebene Anzahl logisch gelöschter Agenten ab dem angegebenen Cursor zu löschen.

Request:

```
DELETE
/api/deletedagents/{mandatorCode}?from={cursor}&count={count}
```

Response:

- Default Return Code: HTTP 200 OK
- Authentifizierung: Standard-Authentifizierung (Nutzername, Passwort und API-Key)
- Autorisierung: Erfordert entweder die Rolle System-Admin oder Mandanden-Admin oder das

```
{
  “results”:
  [
    {
      “cursor”:<string>,
      “agentName”:<string>,
      “deleted”: <bool>,
      “error”: <string>(optinal)
    }
],
  “hasErrors”: <bool>,
  “nextCursor”: <string|null>
}
```

  Function-Right ModifyAgent
- Request Parameter:
  - From: optional, gibt den Cursor an, ab dem das Löschen beginnen soll. Wird der Parameter nicht übergeben, beginnt das Löschen am Anfang der Liste. Ein nicht existierender Cursor führt zu einem 404 Fehler mit ApiObjectNotFound (98002).
  - Count: optional, gibt die maximale Anzahl der zu löschenden Agenten an. Defaultwert ist 50, gültige Werte sind 1 – 50, Ein ungültiger Wert führt zu einem Bad Request Fehler mit ApiInvalidParameter (98000). Auch dieser Defaultwert kann über die SystemConfiguration-Tabelle geändert werden. Siehe Tabelle 3.6.4.

3.6.2 Default return codes

Return-Code	Beschreibung
400	Bad Request Unauthorized (70120), FailedToParseParameters (96000), ApiKeyRequired (96001), ApiInvalidParameter (98000)
401	Unauthorized UserLogOnFailed (19005), UserLogOnBlocked (19006), UserValidityExpired (19012), UserNotYetActivated (19009), ClientIpAddressNotWhitlisted (96003)
403	Forbidden NoAuthorization (70121), ApiKeyRequired (96001), ApiKeyNotFound (96002), MaximumNumberOfCallsExceeded (96004)
404	Not Found MandatorNotFound (13000), ApiObjectNotFound (98002)

Return-Code	Beschreibung
500	Internal Server Error (BusinessException missing)

3.6.3 List of possible agent deletion errors

BusinessExeption	Beschreibung
AgentIdCannotBeNull (11006)	Agent ID darf nicht null sein
AgentReferredByStream (11007)	Agent wird noch von einem Stream referenziert
AgentReferredByJob (11008)	Agent wird noch von einem Job referenziert
AgentReferredByStreamRun (11009)	Agent wird noch von einem Streamrun referenziert
AgentReferredByStreamRunJobProperty (11010)	Agent wird noch von einem Streamrun Job Property referenziert
AgentReferredByRuntimeStreams (11011)	Agent wird noch von einem Runtime Stream referenziert
AgentReferredByRuntimeJobs (11012)	Agent wird noch von einem Runtime Job referenziert
AgentReferredByFileDependency (11043)	Agent wird noch von einer File Dependency referenziert
AgentReferredByExternalDependency (11044)	Agent wird noch von einer External Dependency referenziert
AgentDeletionFailed (11013)	Fehler beim Löschvorgang
AgentReferredByStreamRunJobExecution (11016)	Agent wird noch von einer Streamrun Job Execution referenziert
AgentHeartbeatDeletionFailed (11018)	Agent wird noch von einem AgentHeartBeat referenziert
AgentJournalDeletionFailed (11019)	Agent wird noch von einem Journal referenziert
AgentLogsDeletionFailed (11020)	Agent wird noch von einem Log referenziert
SystemIncidentLogDeletionFailed (65015)	Agent wird noch von einem Incident referenziert
AgentInRunningListeningMode (11022)	Agent ist noch im Listening Mode
AgentHasMasterDataReferences (11051)	Agent wird noch in Masterdaten referenziert
GeneralDBException (80000)	Datenbank Fehler

3.6.4 System Configuration Values

Name	Key	Defaultwert
MaxCountGetDeletedAgents	10000	2000
MaxCountRemoveDeletedAgents	10001	50

3.7 CLI

3.8 Setup

4 Tests

4.1 Checklist

This checklist is processed: ☐

4.2 Developer- and system integration (SIT) tests

As part of the developer and the system integration tests, at least all changes that are described in this document must be tested.

Define tests based on this template.

5 Open questions and problems

Questions are recorded here that are clarified with the specialist department during the creation of the DDD.

Questions	Answer	Clarified with	Date

Table 3 Open questions and problems

6 Checklist for modifications

Select the necessary modifications in the checklist below. Detailed information can be found in the reference.

Area	Modification?
GUI	☐
Journal entries for modification at runtime area	☐
Copy of a job	☐
Tooltips / Preview texts	☐
Full text search	☐
Rights + Visibility	☐
Preparation Script	☐
Termination	☐
Preparation	☐
Execution of a stream run	☐
Journal entries of a execution	☐
CLI	☐
Help texts / Documentation / Handbooks	☐
Utilities	☐
Agent	☐
Logical resources	☐
Installation	☐
Database modifications	☐
Migration	☐
Time change summertime / wintertime	☐
Mobile Apps	☐
Validation (Alphabet und field length)	☐
Insertion Point / Dynamic stream	☐
Reports	☐
Beat files	☐
Dashboard	☐
Setup	☐
Streamworks REST API	☐
Streamworks REST API Testautomation	☐
Web App	☐

Table 4 Checklist for modifications – Overview of necessary modifications

7 Work package planning

The table below documents the rough work packages that need to be carried out to implement the design described above.

Note: A work package is not greater than 16 hours!

Work Package	Effort in h
[Package Description]	X
Sum (Feld rechts mit F9 aktualisieren)	**0**